GCC1734 - INTELIGÊNCIA ARTIFICIAL

Trabalho 2 – Busca Adversarial e Aprendizado por Reforço

Prof. Eduardo Bezerra (ebezerra@cefet-rj.br)

CEFET/RJ
Escola de Informática e Computação
Bacharelado em Ciência da Computação

Outubro/2026

Sumário

1 Introdução **3**

2 Especificações da entrega **3**

3 Forma de avaliação **3**

4 Q1 (3,0 pontos) – Busca Adversarial: Minimax e MCTS **4**

5 Q2 (1,0 ponto) – Q-learning Tabular: Replicação de Experimentos **5**

6 Q3 (1,5 ponto) – Q-learning Tabular: Discretização de Estados **5**

7 Q4 (1,5 ponto) – Q-learning com Aproximação Linear **5**

8 Q5 (1,5 ponto) – Q-learning vs. SARSA: Comparação On-Policy e Off-Policy **6**

9 Q6 (1,5 ponto) – Reward Shaping para Aceleração do Aprendizado **6**

1 Introdução

Este trabalho reúne atividades de busca adversarial e aprendizado por reforço. Na primeira parte, você implementará agentes Minimax e MCTS para um jogo de tabuleiro. Nas atividades seguintes, realizará experimentos com Q-learning tabular e com aproximação linear, além de implementar e comparar outros métodos de aprendizado por reforço.

O código-base está disponível nas pastas `src/adversarial` e `src/rl` do repositório GitHub do curso.

2 Especificações da entrega

Você deve entregar um único arquivo compactado contendo os seguintes itens:

1. A pasta `adversarial`, com os arquivos originais e os arquivos criados ou alterados durante o trabalho.
2. A pasta `rl`, com os arquivos originais e os arquivos criados ou alterados durante o trabalho.
3. Um arquivo `LEIAME.txt` contendo o endereço do vídeo explicativo.

O vídeo pode ser publicado como não listado no YouTube. Informe a minutagem de todos os itens pontuáveis, indicando em que minuto e segundo começa a explicação de cada item. Consulte também as orientações de submissão e apresentação disponibilizadas no material "Apresentação do curso".

Papel do vídeo na correção

No vídeo, espera-se que você explicite as respostas às perguntas do enunciado, demonstre a execução e analise os resultados obtidos, além de descrever as partes relevantes do código.

O nome do arquivo compactado deve seguir o padrão `GCC1734_T2_SEU_NOME_COMPLETO.zip`. Um exemplo: `GCC1734_T2_EDUARDO_BEZERRA_DA_SILVA.zip`. Siga à risca esse padrão de nomenclatura.

3 Forma de avaliação

O trabalho vale 10 pontos, distribuídos entre as atividades a seguir:

Atividade	**Pontos**
Q1: Busca adversarial (Minimax e MCTS)	3,0
Q2: Q-learning tabular: replicação de experimentos	1,0
Q3: Q-learning tabular: discretização de estados	1,5
Q4: Q-learning com aproximação linear	1,5
Q5: Q-learning vs. SARSA	1,5
Q6: Reward shaping	1,5
Total	**10,0**

4 Q1 (3,0 pontos) – Busca Adversarial: Minimax e MCTS

Nesta atividade, você projetará agentes adversariais usando os algoritmos de busca Minimax e MCTS. O código-base está disponível na pasta `adversarial` do repositório GitHub do curso.

Você deverá adaptar os algoritmos **Minimax** e **MCTS** para um novo jogo de tabuleiro com complexidade comparável ao *Connect Four*, apresentado em aula, e implementar agentes capazes de jogar de forma autônoma.

Jogo a ser implementado

Você deve considerar **obrigatoriamente um** dos jogos listados a seguir:

1. Gomoku (Five in a Row);
2. Othello (Reversi);
3. Quarto;
4. Breakthrough.

As regras podem ser consultadas nos endereços indicados. O jogo a implementar é determinado pela matrícula. Se m é a sua matrícula, use:

$$\text{ID}(m) = [(\text{parte numérica de } m) \bmod 4] + 1.$$

Por exemplo, o aluno com matrícula $m = 1223387\text{BCC}$ deve implementar o jogo de identificador 4 (Breakthrough), pois $1223387 \bmod 4 + 1 = 4$.

Tarefas da Q1

1. Implementar as regras e a lógica do jogo;
2. Desenvolver um agente baseado em Minimax com profundidade limitada e função heurística de avaliação;
3. Desenvolver um agente baseado em MCTS com número fixo de iterações;
4. Criar uma interface interativa em **linha de comando**, com visualização clara do tabuleiro;
5. Permitir que um jogador humano possa jogar contra os dois agentes.

Evidências exigidas na Q1

No vídeo explicativo, apresente as regras do jogo e sua implementação, a heurística usada no Minimax, a política de simulação do MCTS, uma comparação qualitativa entre os agentes e observações sobre desempenho, limitações e possíveis melhorias.

5 Q2 (1,0 ponto) – Q-learning Tabular: Replicação de Experimentos

Utilize a classe `QLearningAgentTabular`, já fornecida, e repita os experimentos de treinamento e teste nos seguintes ambientes:

- Blackjack;
- Cliff Walking;
- Frozen Lake.

Evidências exigidas na Q2

Para cada ambiente, informe os hiperparâmetros usados no treinamento e apresente, no vídeo, a execução e a análise dos resultados de treinamento e teste.

6 Q3 (1,5 ponto) – Q-learning Tabular: Discretização de Estados

O Q-learning tabular requer conjuntos de estados e ações finitos e enumeráveis. No ambiente `MountainCar-v0`, as ações são discretas, mas o estado é contínuo: posição e velocidade assumem valores reais. Para representar esses estados na tabela $Q(s, a)$, implemente a discretização especificada a seguir:

1. Divida a posição e a velocidade em um número fixo de intervalos (*bins*);
2. Mapeie os estados contínuos para representações discretas;
3. Adapte o Q-learning tabular para treinar o agente nesse espaço discretizado.

Evidências exigidas na Q3

No vídeo explicativo, apresente e explique o código modificado e os resultados do treinamento. Como referência, consulte também o vídeo no YouTube: Use Q-Learning to Train Gymnasium MountainCar-v0.

7 Q4 (1,5 ponto) – Q-learning com Aproximação Linear

Repita os experimentos da Seção 5, utilizando agora o algoritmo Q-learning com aproximação linear, implementado na classe `QLearningAgentLinear`. Nesse caso, a função de valor $Q(s, a)$ é representada como combinação linear de funções características (*features*) do estado e da ação:

$$Q(s, a) = \sum_i w_i f_i(s, a),$$

em que cada w_i é um peso ajustado a partir do erro temporal (δ):

$$w_i \leftarrow w_i + \alpha\, \delta\, f_i(s, a), \qquad \delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a).$$

Tarefas da Q4

- Definir funções características $f_i(s, a)$ capazes de representar informações relevantes do ambiente. Comece com características simples (por exemplo, *bins* para posição e velocidade no MountainCar) e refine-as iterativamente (por exemplo, adicionando indicadores de risco ou de proximidade do objetivo);
- Explicar o significado de cada característica e como os refinamentos tratam limitações das representações iniciais;
- Treinar o agente e analisar a evolução de $Q(s, a)$, dos pesos aprendidos e da generalização para estados não visitados.

Evidências exigidas na Q4

No vídeo, apresente e discuta os resultados, compare o desempenho com o Q-learning tabular e analise os compromissos envolvidos, como o efeito do número de características sobre generalização e sobreajuste.

8 Q5 (1,5 ponto) – Q-learning vs. SARSA: Comparação On-Policy e Off-Policy

Implemente um agente baseado em SARSA (on-policy) usando a classe `QLearningAgentTabular` como base. Compare seu desempenho com o Q-learning (off-policy) nos ambientes da Seção 5: Blackjack, Cliff Walking e Frozen Lake. SARSA aprende o valor da política realmente executada, incluindo a exploração, enquanto Q-learning assume comportamento ótimo futuro.

Evidências exigidas na Q5

No vídeo explicativo:

- Explique as diferenças nas regras de atualização: SARSA usa $Q(s', a')$, em que a' é a ação escolhida, enquanto Q-learning usa $\max_{a'} Q(s', a')$;
- Apresente resultados comparativos, como recompensa média, convergência e trajetórias no Cliff Walking;
- Discuta quando cada algoritmo é mais apropriado, por exemplo, SARSA em ambientes de risco, como robótica, e Q-learning em simulações.

9 Q6 (1,5 ponto) – Reward Shaping para Aceleração do Aprendizado

Implemente *reward shaping* baseado em potencial para acelerar o aprendizado em ambientes com recompensas esparsas. Use uma função potencial $\Phi(s)$ (por exemplo, baseada na distância ao objetivo) para ajustar as recompensas:

$$r'(s, a, s') = r + \gamma\Phi(s') - \Phi(s).$$

Teste a abordagem nos ambientes da Seção 5 ou da Seção 7, comparando o aprendizado com e sem *shaping*.

Evidências exigidas na Q6

No vídeo explicativo:

- Explique a função potencial e apresente exemplos numéricos, por exemplo, no Cliff Walking;
- Mostre a aceleração do aprendizado, como a redução do número de episódios até a convergência, e discuta a integração com as características;
- Analise limitações e situações em que o *shaping* pode falhar ou distorcer a política ótima.